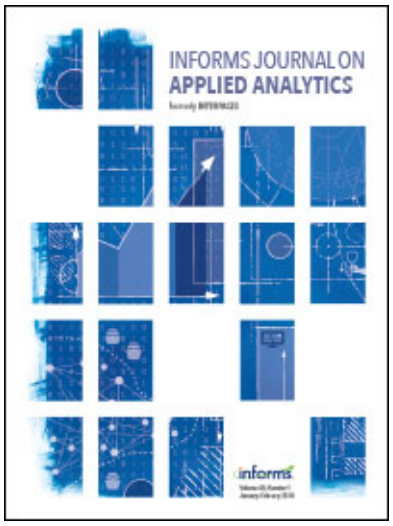


INFORMS Journal on Applied Analytics

Publication details, including instructions for authors and subscription information:
http://pubsonline.informs.org

Planning Courses for Student Success at the American College of Greece

Ioannis T. Christou, Evgenia Vagianou, George Vardoulias

To cite this article:
Ioannis T. Christou, Evgenia Vagianou, George Vardoulias (2024) Planning Courses for Student Success at the American College of Greece. INFORMS Journal on Applied Analytics

Published online in Articles in Advance 04 Mar 2024

. https://doi.org/10.1287/inte.2022.0083

https://pubsonline.informs.org/journal/ijaa

INFORMS JOURNAL ON APPLIED ANALYTICS
Articles in Advance, pp. 1–15
ISSN 2644-0865 (print), ISSN 2644-0873 (online)

Planning Courses for Student Success at the American College of Greece

Ioannis T. Christou,[a,*] Evgenia Vagianou,[a] George Vardoulias[a]

[a] Department of Information Technology, The American College of Greece, 15342 Athens, Greece
*Corresponding author
Contact: ichristou@acg.edu, https://orcid.org/0000-0002-0206-6973 (ITC); jes@acg.edu, https://orcid.org/0009-0000-4912-3106 (EV); gvardoulias@acg.edu, https://orcid.org/0000-0002-4361-4032 (GV)

Received: October 24, 2022
Revised: February 22, 2023; June 21, 2023; August 14, 2023; November 20, 2023
Accepted: December 7, 2023
Published Online in Articles in Advance: March 4, 2024

https://doi.org/10.1287/inte.2022.0083

Abstract. We are concerned with the personalized student course plan (PSCP) problem of optimizing the plan of courses students at the American College of Greece will need to take to complete their studies. We model the constraints set forth by the institution so that we guarantee the validity of all produced plans. We formulate several different objectives to optimize the resulting plan, including the fastest completion time, course difficulty balance, and maximization of the expected student grade point average given the student's performance in passed courses. All resulting problems are mixed-integer linear programming problems with a number of binary variables, that is, the max number of terms times the number of courses available for the student to take. The resulting mathematical programming problem is solvable in less than 10 seconds on a modern commercial off-the-shelf PC, whereas the manual process used to take more than one hour of advising time for every student and, as measured by the objectives set forth, resulted in suboptimal schedules.

History: This paper was refereed.

Keywords: course planning • mixed-integer programming • multiobjective optimization • data mining • course grade estimation

Introduction

The American College of Greece (ACG) is a nonprofit higher-education institution, the oldest American-accredited college in Europe and the largest private college in Greece. Deree College is one of ACG's major divisions, which facilitates undergraduate and graduate education.

Students enrolled at Deree College are encouraged to seek advice from their academic advisors to plan for their course of action while studying; in their first two years, the students get advice from the advising office that focuses on their study plan for the forthcoming term. After this period, they are directed to their major department, and the department head (DH) and/or program coordinator (PrC) become their main source of advice for the remainder of their studies. This practice results in significant pressure on DHs/PrCs, who, besides their normal teaching, scholarly, and administrative duties, are also preparing customized study plans for each of their advisees. Acting as a *program advisor*, the DH/PrC comes up with a *personalized study plan* that must obey complex constraints (details in Appendix A) and optimize several objective criteria based on student preferences. Because manual optimization for such a problem is practically out of the question, advisors often resort to predefined templates of feasible study plans that they tamper with to reflect the student's status (progress, grade point average (GPA), etc.) and "look good enough" for the student.

This heuristic process takes up a significant amount of time for DHs/PrCs; each student study plan usually takes more than one hour of intense study. It is estimated that the academic advice provided by both the advising office and the department, on an annual basis, translates to more than 3,000 hours of work.

We took up the challenge of automating this process by developing the *ACG Individual Student Course Plan Optimizer* (SCORER), a small application that, when fed with current student data and desires, produces a feasible study plan that obeys all relevant constraints (common college policy and program-specific requirements) while optimizing individual student objectives. Through the graphical user interface (GUI), SCORER accepts student data as input and reads program-specific data through configuration files. The first includes (a) courses the student has completed, (b) the student's preferences regarding course options and electives, (c) student choices regarding registration during summer periods, (d) max desired number of courses per term, and (e) student priorities over the objectives implemented in the program; program-specific data include (f) specifics about every course of

the program, (g) program focus areas, which may involve complex subset-selection constraints, and (h) other constraints. SCORER creates a complex mixed-integer linear program (MILP) (Wolsey and Nemhauser 1988, Christou 2011, chapter 1) and calls GUROBI (Gurobi Optimization LLC 2022a), one of the fastest commercial optimization solvers, to solve it. Once GUROBI finds an optimal solution, SCORER translates the optimization problem variables back to the problem domain variables and presents to the student a corresponding schedule of courses in a human-readable format. An edit area allows the user (student or advisor) to specify the desired term for any planned course or reject a proposed course and then reoptimize based on the new input. The user can further customize the study plan by specifying constraints related to the number of courses for any particular term or set of terms.

Related Work

There is extensive published work on higher education *curricular optimization* (see, for example, Chiarandini 2012, Unal and Uysal 2014, Kassa 2015, Strichman 2017, Gonzalez et al. 2018, Heileman et al. 2019, Thompson-Arjona 2019) but not specifically on computing personalized individual student course plans. Academic curricular optimization is primarily concerned with the problem of course scheduling while obeying institutional constraints.

Variations on the theme abound. Thompson-Arjona (2019) is aiming for the design of a more balanced curriculum for electrical engineering and attempts to avoid "toxic combinations" by focusing on course offerings. The formulated prerequisite and corequisite constraints apply only to mandatory courses of the curriculum, whereas the course options are not considered. In the same spirit, Chiarandini (2012) presented a generalized version of the balanced academic curriculum problem (BACP), an NP-hard problem that has been heavily researched by Castro and Manzano (2001). The proposed generalized version of the problem (GBACP) introduces choice in students' plans through a set of preformed curricula alternatives, which lead to the fulfillment of the degree requirements. In their computational results, Chiarandini (2012) present solutions for problems with up to 37 different curricula. For comparison purposes, note that there are 546,480 different combinations of courses an information technology (IT) major student at Deree can choose from, without taking into account the variety of liberal education (LE) course options.

Monette et al. (2007) have presented an extensive study of the original BACP, which does not consider elective courses. The focus is on the objective function that attempts to balance the load across the study periods. Hnich et al. (2002) tackled the BACP with constraint satisfaction problem (CSP) techniques. Strichman (2017) considers the joint problem of automating course, classroom, and exam scheduling for the industrial engineering faculty at Technion, Israel. He developed TieSched, a scheduling system that comprises several components for solving classical Satisfiability (SAT) problems, including CSP. Gonzalez et al. (2018) developed an integer programming formulation that generates a course schedule using the repeated-week format. They use cadet registration information to determine the number of sections per course, which will ensure cadet on-time graduation while accomplishing mandatory military training.

In general, there is a large body of software concerned with timetabling of courses; see, for example, Comm and Mathaisel (1988) for an early discussion on class and calendar scheduling, Miranda (2010) for a more recent study, or online web apps.[1] A review of approaches for solving the university course timetabling problem was presented in Babaei et al. (2015) and again more recently in Ceschia et al. (2023). The described approaches include exact techniques such as mixed-integer programming (MIP) and max CSP as well as metaheuristics such as genetic algorithms (GA) and evolutionary algorithms (EA), variable neighborhood search (VNS), harmony search (HS), and multiagent systems (cooperative search). An older approach appears in Schaerf (1999). Closest to our work is the research presented in Bowman (2021) regarding the problem of developing near-optimal student study plans in that individual student needs are taken into account and long-term plans are created. Nevertheless, the main focus of this work is on the student registration needs for the immediate next term in light of class capacity restrictions, which relates to the number of sections/cohorts scheduled for each course. Class capacity is also the driving force behind the work of Garcia (2019).

There are major differences between timetabling and the personalized course study plan problem that we are concerned with. The timetabling problem deals mostly with assigning timeslots of the week to course sections/cohorts and sections to available classrooms. Although not in the scope of our current study, in Appendix A, we show how knowledge of course timeslot assignments can be incorporated in an extension of our current model, which decides the particular course section that a student may register for during a term to avoid time conflict between selected courses. The commercial system Prepler[2] has many of the features that SCORER has and a lot more analytics/dashboard features but does not clearly involve the student with the produced degree plan objectives.

In the rest of the paper, we first present a description of the *personalized student course planning problem* at

Deree College without resorting to mathematical expressions (fully described in Appendix A); we then present the details of our program implementation, and subsequently, we present data regarding the efficiencies achieved by the adoption of the system by the IT and the cybersecurity and networks (CYN) academic programs, hosted by the IT department; recently, the system was also installed and is currently being tested by the psychology (PSY) and communications (CN) programs. Finally, we discuss the impact of our work, and in the last section, we present our conclusions and future directions.

Problem Description

The focus of our study is on developing effective (GPA-oriented) and efficient (completion time–oriented) study plans while adhering to the policies posed by Deree ACG (constraints) that directly or indirectly affect the process. The three major dimensions that we consider are (a) course offerings, (b) term course load, and (c) course specifics. In this section, we present the relevant concepts as well as the nature of the constraints imposed by the practices and policies of the institution; their exact formulation is deferred to Appendix A.

Concepts

Each course has an associated number of credits. We also associate a nonnegative difficulty level that is estimated via the student failure rates in the past three years or via the average grade of the student cohorts of the past year (for recently introduced courses) or the instructor's "expert guess" for new courses. The difficulty is defined to be zero for courses with failure rates not exceeding 10% on average during the past three years and goes up to 10 proportionately as the failure rate average goes up to 100% (which never happens); for courses that are taught less than three years, the difficulty level is zero when the average grade of the students in the class is 3.5 or higher (max being 4.0 in the U.S. system) and goes up to 10 proportionately as the average grade drops to zero.

Academic Terms and Course Load Constraints

The UG programs utilize five terms, that is, Fall, Spring, Summer1, Summer2, and SummerTerm. Summer1 and Summer2 are fast-paced four-week terms that run during June and July, respectively, whereas SummerTerm lasts eight weeks starting at the end of May and running until the last week of July. SCORER makes use of the linear succession of terms to ensure coherent course sequence. The only exception is the SummerTerm, which overlaps with Summer1 and Summer2 and therefore works as an alternative time-step between Spring and Fall. Moreover, following college policy, students have the option to exclude any or all three summer terms. For example, a student may register for one or two courses in Summer1 and/or Summer2 terms without registering for SummerTerm term.

Based on this, we impose the following numbering of the terms of any academic year: Fall→1, Spring→2, Summer1→3, Summer2→4, SummerTerm→5.

The policy/constraints regarding term course load are described in Appendix A.

Curriculum Constraints

The UG programs consist of two main components that pose significant constraints, liberal education and concentration, and general electives.

The liberal education component requirements involve specific disciplines such as social sciences, humanities, and fine arts and allow choice among several liberal education–labeled courses from every area. Along with the general electives, these courses are, on average, less demanding than concentration courses, are often used to balance the anticipated work per term, and may be completed at any time during the student's studies.

Course levels, associated with the Level-k courses denoted by $L_4, L_5,$ and L_6 in Table A.1, rule the concentration component of the curriculum to ensure specialization maturity progress, resulting in the following complex (at first glance) constraints:

- To register for Level-5 courses, students must have completed at least four Level-4 courses.
- To register for Level-6 courses, students must have completed all Level-4 courses *and* 4 Level-5 courses.

All courses are subject to *prerequisites/corequisites*, which ensure the necessary background for a course and reflect one of the most important sets of constraints to obey. Students cannot enroll in a course unless they have completed all its prerequisites (if any exist). Course corequisites are course requirements that the student must have either completed before registering for the course or register for the course simultaneously with its corequisites.

While working on this problem, with two of the authors being "problem owners," we realized that there are courses with no typical prioritization (e.g., prerequisites) that, if taken at a certain time during their studies, can have a notable impact on the students' performance as they increase the overall concept understanding and maturity. To tackle such cases, we considered pairs of courses and implemented a soft-order precedence constraint between them to indicate that if the student plan proposes both courses and the second one is not taken yet, then they should be taken in the specified order.

Objective Functions

After discussing with advisors and students, we deduced three major objectives for SCORER, namely, completion time of the produced plans, balancing the academic difficulty per term, and planning for a high overall GPA. The most common objective from a student's perspective is to maximize expected GPA, the ultimate metric of success for bachelor's degrees, followed by the completion/graduation time.

Maximizing the Expected GPA via AI

We would like to be able to come up with course plans whereby the expected student GPA would be maximized given the courses we suggest. Although effective to a certain extent, the difficulty-level approach is a purely static one that does not consider individual student performance in classes taken so far. Course difficulty levels are usually associated with class failure rates rather than actual grades, and the related objective is to distribute challenging mandatory classes throughout the plan.

Having access to fully anonymized student records, we developed a machine learning/data mining–inspired approach whereby we extract, from the data set containing all student course records, the set of all quantitative association rules that hold on the data set with min user-specified support and confidence that are in one of the following three forms:

(a) if the student's grade on a course is at least equal to a threshold value, then the student's grade in a different course is expected to be at least equal to another specified threshold (performance on a certain course predicts min performance on another course)

(b) if the student's grades on two specified courses are at least equal to two certain threshold values, then the student's grade in a different course is expected to be at least equal to another specified threshold (performance on two courses predicts min performance on another course)

(c) if the student's grades on three specified courses are at least equal to three certain threshold values, then the student's grade in a different course is expected to be at least equal to another specified threshold (performance on three courses predicts performance on another course)

These statements are mathematically specified in Appendix A.

We extract this ruleset with minimal support threshold set to 0.5% and minimal confidence threshold set to 90% using QARMA (Christou et al. 2018, Christou 2019), a highly parallel/distributed tool that has given excellent results in diverse applications (see Christou et al. 2022 for applications in predictive maintenance and the fashion industry). These rules predict student success in courses based on student performance in completed courses. Common experience says that there are courses that share ways of concept perception. For example, a student who obtained a high grade in calculus is likely to get a high grade in applied math; similarly, one who gets a high grade in an operating systems course is likely to also get a high grade in a high-performance computing course. For each student, we estimate the grade in a course as the highest value predicted by any rule in the ruleset that has the course as its consequent and that fires given the student's course history. If no rule fires for a particular course, as expected to happen in the case of freshmen, we do not assign a predicted grade, and such courses are not further "promoted" for inclusion in the student plan. Thus, the scheme applies to students in their sophomore year or later. Moreover, the rules do not entail knowledge of external factors that may have an impact on students, for example, a change in teaching faculty. Such "concept drift" problems can be countered by regular reruns of the rule extraction algorithm.

The exact objective function and related constraints are fully defined in Appendix A. The idea of using past data of student performance in certain classes to predict performance in future classes in the context of personalized study plans also appears in Takada et al. (2013), where the authors only utilize classical statistical methods such as correlation analysis. A more sophisticated approach appears in the more recent PhD study of Slim (2016), where he considers Bayesian belief networks (BBN); the thesis, however, is about curricular optimization and not individual study plans; therefore, there is no attempt to use BBNs to recommend particular elective courses to individual students.

This rule-based approach cannot be easily replaced by popular machine learning (ML) techniques such as (deep) neural networks (Gutierrez-Rojas et al. 2022). This is because of the nature of the data set, which is very sparse: every student registers for up to 40 courses, whereas there are more than 180 courses available to the (IT or CYN) student to choose from. This implies that if the grade of a student in a course is a feature, the cells of every row in the data set would be more than 75% empty. Neural networks often do not handle missing values well, even with the advent of imputation (Lin and Tsai 2020) and generative adversarial network (GAN)–based learning techniques (Goodfellow et al. 2014).

Problem Complexity

In the same way that the GBACP is reduced to BACP that is shown to be NP-hard (see Chiarandini 2012), we have the following:

Theorem 1. *The personalized student course plan (PSCP) problem is NP-hard.*

The proof is available in Appendix B.

System Implementation

System Design

The plan optimizer has the form of a small-scale Java desktop application (approximately 6,000 lines of code [LOC]), which uses the GUROBI Java application programming interface (API) (Gurobi Optimization LLC 2022b) to communicate with the GUROBI optimizer and the Java Swing API for GUI development. It accepts user input, creates the mathematical programming model with the combined objective function (A.25) and subject to Constraints (A.1)–(A.24) as specified in Appendix A, produces an LP-format file, and then calls an optimization solver (currently GUROBI) to solve the problem specified in the LP file. The whole codebase is available as open source from the first author's GitHub repository.

User Interfaces

Main Planning User Interface. The main GUI is used for student/advisor input and display of the resulting proposed study plan. Figure 1 shows an optimization run for a freshman student planning (during the summer) to take the first course in the fall 2022 semester.

Schedule Editing. The outputs area presents the full course schedule as computed by the optimizer's solution to the model and includes an editor window under the heading "Edit Time-Slots for Courses" to facilitate user customization of the produced study plan. The user can indicate a different preferred term for any scheduled course, simply reject the course from the proposed term by entering a minus sign (−), or set constraints on the number of courses the user wishes to take on any scheduled term by entering an inequality such as "<=3" to specify, at most, three courses for the specific term.

Administrator Interfaces. We employ two user interfaces to help program advisors create the data files needed by the application. The produced data files are stored in an auxiliary subfolder within the application root folder; after the program advisor has created all necessary course and constraint-related data files, the application is ready for use by the advisors and students alike and is distributed by the College Information Resources Management Department as a zip archive. The same department is responsible for installing the GUROBI solver on the laptops of faculty and students alike.

The first user interface guides an administrator to manage course-related data (add/edit/delete courses), shown in Figure 2. This minimalistic interface utilizes a simple card with information about one course at a

Figure 1. (Color online) SCORER GUI

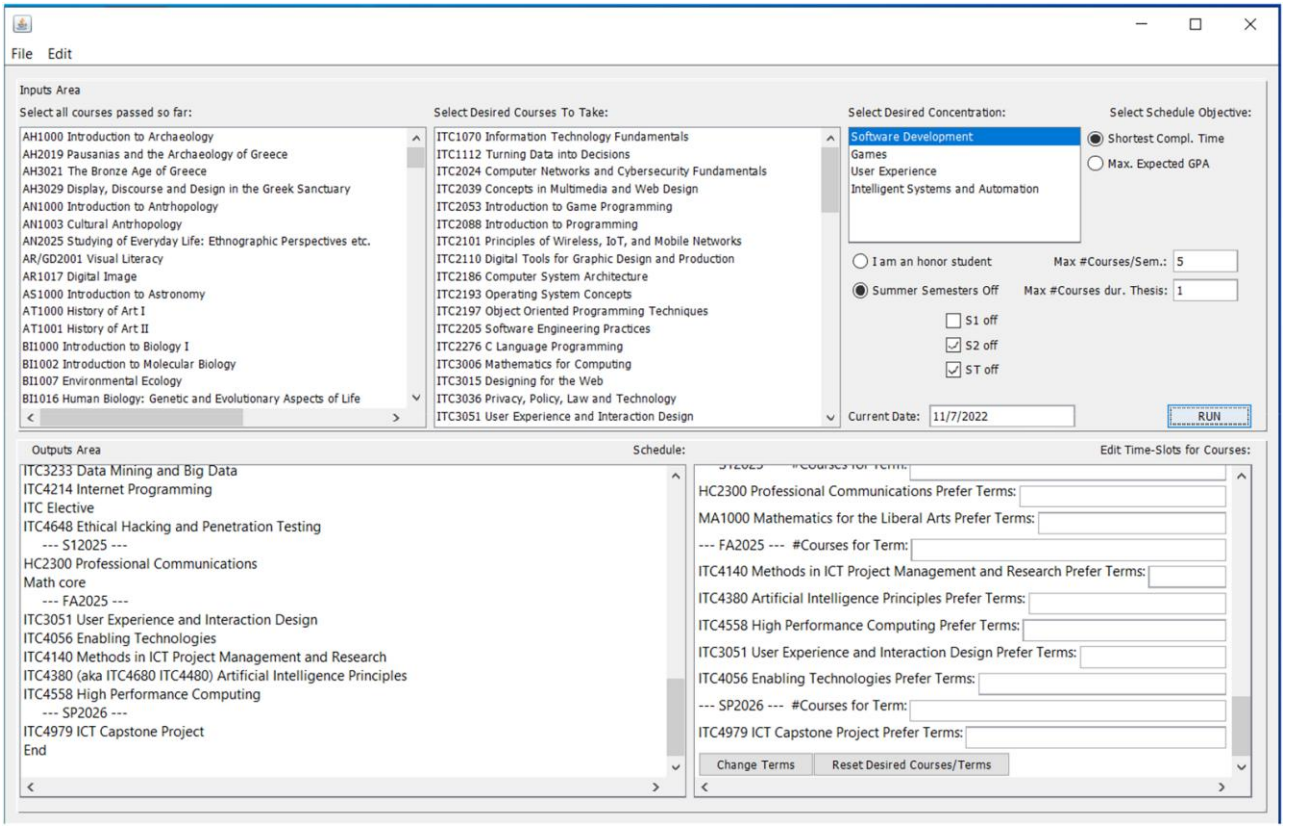
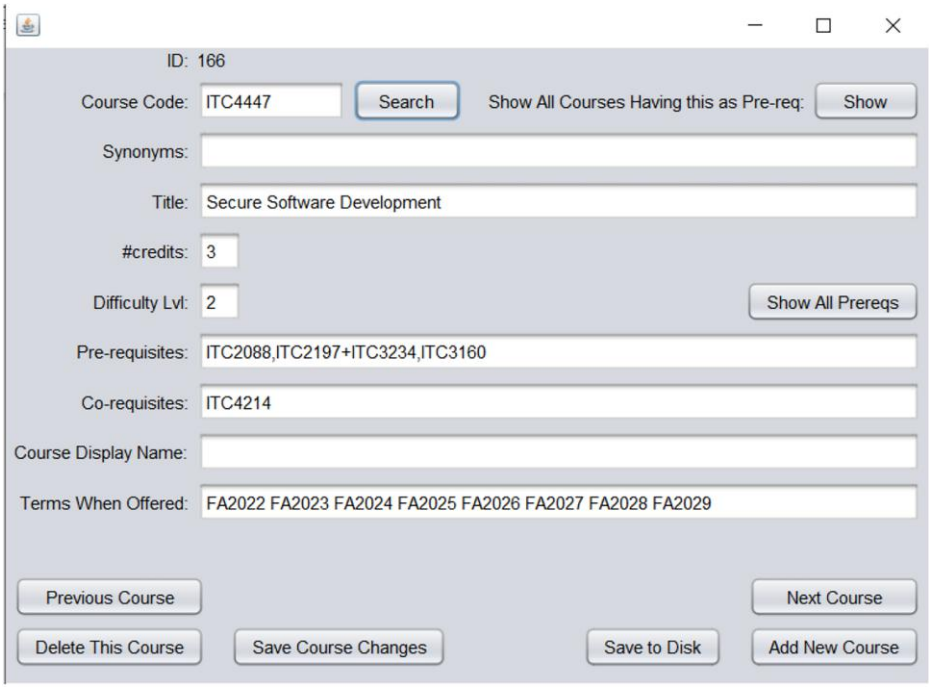


Figure 2. (Color online) Course Editor GUI



time, allowing the user to edit the information currently being displayed.

The second administrator interface helps authorized users create and edit course groups, which are used in defining the constraints that any valid study plan must obey. In Figure 3, the selected group "LE-core" contains five courses, all of which must be taken by the student (indicated by setting the "#Courses Constraint Value" text field to five).

Results

The IT program at Deree offers a total of 69 information technology and communications (ITC) courses, whereas the total number of courses available to the students exceeds 180. Planning for freshmen who have yet to take their first class results in an MIP with three continuous variables (shown in the objective function), 7,854 binary variables, and a total of 36,253 constraints. The CYN program requires students to take 24 area-specific courses

Figure 3. (Color online) Course Group (Constraints) Editor GUI

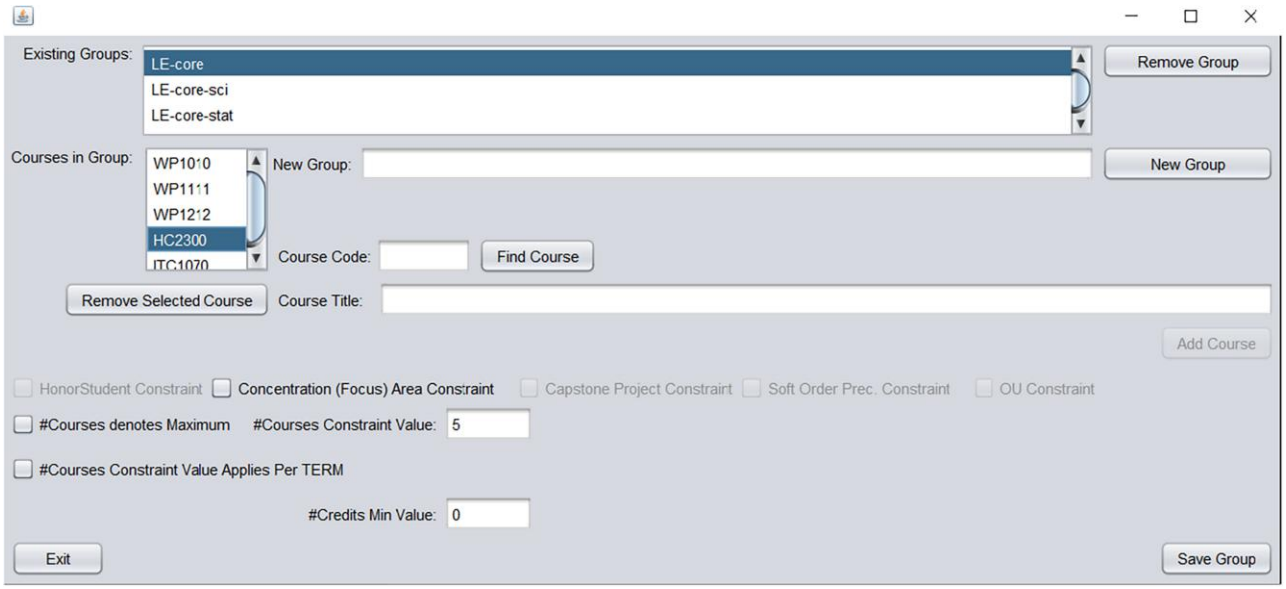
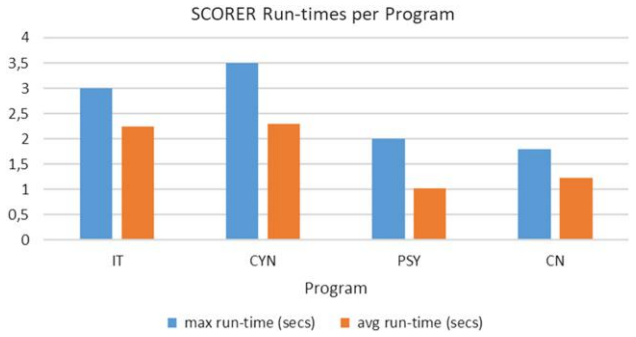


Figure 4. (Color online) SCORER Average and Max Run-times for Different Programs



and another two selected out of a group of five ITC courses, with the total number of courses available to students exceeding 180. Such models are solved by GUROBI 9.5.1 on a laptop equipped with a third-generation Intel Core i7 CPU with 16 GB RAM in less than 10 seconds, making the program respond almost in real time to the user requests and thus allowing for extensive schedule editing and sensitivity analysis afterward.

Figure 4 provides some data on the time it takes to produce useful study plans on such a laptop for the IT, CYN, PSY, and CN programs. The max and average runtimes for each program are obtained from 10 samples for the IT program, 8 samples for the CYN program, and 4 samples for the CN and PSY programs; the difference in sampling is because of the variety of concentration areas in each program. The exact times were obtained using the Java system called System.currentTimeMillis(). As can be seen, the runtime never exceeds 3.5 seconds of wall-clock time; in fact, the more constrained the student schedule (by courses already passed), the easier it becomes for the model to be optimized.

Given the consistent response times of the system, it is estimated that the program advisors advising the IT and CYN undergraduate student body have reduced their load by more than three orders of magnitude (from approximately 3,600 seconds/student to <3.5 seconds/student). Equally important is that the system can propose a plan of courses to the student that maximizes that student's expected GPA, which is a major key performance indicator of measuring student success at the undergraduate level, something practically impossible if attempted manually for individual students because of the enormous amount of effort required.

System Responsiveness Under Open-Source MIP Solvers

As seen in Figure 4, the system responds very quickly to user commands, making it a highly interactive tool that can be used to edit schedules very quickly, at least when the MIP problem is given to the state-of-the-art commercial solver GUROBI. We have also tested the program with the open-source MIP solver SCIP.[3] SCIP is a viable alternative, always managing to find an optimal solution to the problem, though the runtime makes SCORER less interactive than it is when GUROBI is used as the underlying solver; still, on a modern Intel Core i9-10920X CPU, SCIP solved the hardest problem in our test set in less than 10 seconds but took approximately 30 seconds to solve the same problem on the aging laptop. For this problem, it is thus possible to trade the algorithmic sophistication of a commercial solver with hardware (CPU only) advances (neither solver utilizes the GPU of any machine).

Discussion

We now turn to the impact our work has had on the college procedures and the people using the software so far. At ACG, the problem of advising students as to what classes to take during their studies has been a manual, labor-intensive process, taking an estimated toll of approximately 3,000 hours of human labor of faculty and personnel high up in the hierarchy every year. This time estimate does not consider costs associated with program advisor fatigue from a complex task that requires their full attention or student anxiety arising from the inevitable delays in getting an updated plan of studies. Even more, the cost of possible errors that are hard to avoid in a manual process, which then require further corrective action, is not considered.

Outside of the technology-related programs, we presented SCORER to the DHs and PrCs of the School of Liberal Arts and Sciences (SLAS); we showcased SCORER more extensively to

- the DH of the English and modern languages department, which hosts the English academic program with an enrollment of 69 students,
- the DH of the communications department, which hosts two programs (communications and cinema studies) with a total enrollment of over 200 students, and
- the DH of the psychology department, which hosts the psychology program, enrolling 600+ students.

The English department head mentioned that "I expect the app to be instrumental for advisors and save them time by involving a student in his/her study plan. Also, a new student would take responsibility and plan out his/her study by selecting the respective filters to customize preferences and have a working plan that may be adjusted as needed."

The communications DH commented the following: "I believe this platform could prove a valuable tool for both academic advisors and students. My first impression was very positive. The platform gives students the opportunity to plan ahead and to calculate and recalculate their semesters as they need. It will give them a sense of control over their studies, without feeling that

they constantly need an advisor to tell them what to do. Pedagogically speaking, this on its own deserves consideration." In fact, this is the feeling that IT and CYN students, who have been introduced to the use of SCORER, also expressed. They stressed the importance of having meaningful control over their study plan.

The head of the psychology department had a very positive feeling about SCORER and said in her own words: "My first impressions of the platform are very positive. SCORER has an intuitive and user-friendly interface. Simple and contemporary to use but also feature-rich. It seems a very valuable tool for (a) academic advisors to efficiently and swiftly generate a study plan for each student, taking under consideration a number of parameters, such as course offering and availability, duration of studies, desirable number of courses taken per semester, etc.; (b) students to easily organize their schedules, but most importantly, take responsibility of their own study plan and be independent and informed learners; and (c) department heads to obtain the data from students' study plans and develop academic schedules based on needs and demands. Knowing how many students need to register for courses per semester, they can manage course limits and number of course sections accordingly. And [one] final comment that impressed me is that the platform can easily adapt and modify the study plan if needed."

Indeed, another important positive side effect of the introduction of SCORER to the college operations arises from the fact that the student plans, besides being finalized in no time, can also be transmitted from SCORER and centrally collected in order to effectively estimate the number of sections needed for each course throughout the next terms, thus supporting optimal construction of the program course rotation and scheduling. The current implementation of this process often leads to suboptimal results, with students being unable to register for the courses they had planned. The introduction of SCORER and the centralization of all student course plans will inevitably lead to far better estimates of cohort size for each course for the next years and subsequently to many fewer disruptions of students' study plans.

At first demonstration, the registrar and the dean of the School of Business Administration (SBA) appeared skeptical in view of employing SCORER. The registrar expressed concerns regarding duplication of procedures that are supported by the existing system that the college's registrar's office is using. The dean of SBA expressed concerns regarding the freedom that such a system would leave to the students for a personalized study plan.

Following discussion, the objective and role of SCORER became clear to the administration. On a relevant question of one of the coauthors, the registrar suggested that the support of the existing system is limited to a listing of pending courses for the completion of the degree. It became clear that the objective was to use SCORER to complement existing procedures by providing a recommendation upon which students and advisors could act.

The first impression of the director of the advising office further supports the expected impact of the use of the tool on the effectiveness of the advising process. She particularly commented on the opportunity of improving the advising service.

Using modern advances in optimization software and modeling, we have managed to reduce the program advisors' workload by offloading the problem to computers that can solve it almost instantaneously. The response times of the software, together with the course editing user interfaces we have implemented, allow students to use the software themselves and edit the resultant proposed schedules as many times as they wish until they get the study plan that feels exactly right for them. Utilizing advanced data mining techniques, we aspire to get accurate estimates for the grades students are going to get in a course before they even register for it and use it to maximize students' expected GPA.

Conclusions and Future Directions

The system is currently in use by the information technology and the cybersecurity and networks programs at Deree College of the ACG. Our short-term plans include deploying SCORER for use by most programs offered by the SLAS. SCORER has already started being evaluated on a pilot basis by the psychology and the communications programs. Gradually, we will introduce the software to the rest of the schools.

Two PrCs suggested that SCORER should consider the time offerings of future courses so that the resulting proposed study plan is compatible with the time offerings of the courses in all future terms. Although this preassumes knowledge of long-term course timetables, in Appendix A, we show how such constraints can easily be incorporated into the model if the timing of every course section in the future is known in advance.

Based on the feedback of program coordinators, we plan to investigate multiple dimensions of the courses' difficulty factors to better balance study plans. We also plan to further improve the course grade predictor and test its limits to estimate student success in a given program before the student is even admitted to that program.

Appendix A. Model Formulation

In this appendix, we model all objectives and constraints used in our problem. The objectives developed and discussed previously are not mutually exclusive, in fact, given student priorities can be easily combined in one single objective, according to a linear objective function with coefficients

b^G, b^D, and b^{DL} for each of the three objectives, namely, expected GPA, length of time to completion, and max difficulty imbalance between any two terms, forming a triobjective function.

We have concluded it is more convenient to simply ask students to choose what they consider their most important criterion for course planning and prioritize the two remaining objectives according to a fixed order: when students consider the most important objective to be their expected GPA, the variable associated with the expected GPA is multiplied by the coefficient $b^G = -1,000$ (the overall problem is always a minimization problem), the variable associated with total length of study is multiplied by the coefficient $b^D = 100$, and the variable associated with difficulty balance is multiplied by the coefficient $b^{DL} = 1$; otherwise, when students consider their most important objective the time to complete their studies, we use the coefficient $b^D = 1,000$, the coefficient $b^G = -100$, and the coefficient $b^{DL} = 1$ again.

The values of these coefficients are chosen to mimic a hierarchical prioritization of the objectives.

In Table A.1, we gather all problem parameters that are used in the problem modeling.

The set of decision variables required to model our problem is shown in Table A.2.

Table A.1. Problem Parameters

Symbol	Description	Value range
Sets		
$C = \{c_1, \dots c_N\}$	Finite set of all courses for a program	N/A
LE	Liberal education courses: $LE \subset C$	N/A
Δ_i	Set of LE courses belonging to discipline i: $\Delta_i \subset LE$, $i = 1 \dots \omega$	N/A
L_4, L_5, L_6	Sets of courses belonging to level i, $i = 4,5,6$.	N/A
R_j	Set of courses associated with concentration $j = 1 \dots k$: $R_j \subset C$	N/A
S^d	[optional] Set of desired courses for student to take: $S^d \subset C$	N/A
S^u	[optional] Set of courses the student wishes to exclude from study plan	N/A
I^T	[optional] Set of tuples (i,t) specifying student wishes to take course c_i in term t	N/A
U^T	[optional] Set of tuples (i,t) specifying student does NOT wish to take course c_i in term t	N/A
E	Set of pairs of courses (c_i, c_j) indicating that course c_i is a prerequisite for course c_j	N/A
E'	Set of pairs of courses (c_i, c_j) indicating that course c_i is a corequisite for course c_j	N/A
E^s	Set of "soft-order" pairs of courses (c_i, c_j) indicating that if student takes both in the future, course c_i should be taken before course c_j	N/A
A	Set of courses for which $\hat{g}_c$ is available ($A \subseteq C$)	N/A
Parameters		
N	Cardinality of the set C, $N = \lvert C \rvert$	Natural positive number
$\hat{N}$	Min number of courses required for graduation	Natural positive number
$\bar{c}_i$	Credits for course $c_i \in C$	Natural number
$\bar{d}_i$	Difficulty level for course $c_i \in C$	Real number in [0,10]
L^c	Min number of liberal education credits required for graduation	Natural number
T^c	Min total number of cr required for graduation (121)	Natural positive number
C^{max}	Max number of cr per term the student can register for (17–20)	Natural positive number
$\hat{C}^{rem}$	Remaining number of cr for the student to take to graduate	Natural positive number
S^{max}	Max number of terms constituting a study plan (25)	Natural positive number
T^{max}	Max number of courses student wishes to register for in any term	Natural positive number
Θ^{max}	Max number of courses students wish to register during the term they take their thesis $c_\theta \in C$	Natural positive number
M	Any number ≥ the max number of courses the student is allowed to take in any term (e.g., C^{max})	Real number in $[C_{max}, \infty)$
r_j	Min number of courses from set R_j required for a student who selects concentration j	Natural number
N^d	Min number of different LE areas the LE courses must come from	Natural positive number
$o_{i,t}$	Indicator parameter in {0,1} indicating if course $c_i \in C$ will be taught in term $t = 1, \dots S^{max}$	Number in {0,1}
$\hat{g}_c$	[optional] Estimated grade of student on course c	Number in [0,4]
g^{thres}	[optional] User-specified threshold below which an estimated grade does not enter the set A (2.5)	Number in [0,4]
b^G	Coefficient related to the GPA maximization objective	Number
b^D	Coefficient related to the shortest time completion objective	Number
b^{DL}	Coefficient related to the term difficulty level minimization objective	Number

Note. cr, credits.

Table A.2. Model Decision Variables

Symbol	Description	Value range
x_i	Indicates whether course $c_i \in C$ is included in the study plan	{0,1}
$x_{i,t}$	Indicates whether student registers for course $c_i \in C$ in term $t = 0, 1, \dots S^{max}$, with $t = 0$ indicating course has already been passed by student	{0,1}
z_i	Indicates whether the LE discipline Δ_i is covered	{0,1}
D	Denotes the last term the student registers for any course before graduation	$\mathbb{N}$
D^L	Max sum of course difficulty levels during any term	$\mathbb{N}$
G^e	Max sum of estimated grades the student accumulates under proposed study plan	Number in $[0, 4\hat{N}]$

The Model

The full model has as follows:

$$\min C(D, D^L, G^e, x, z) = b^G G^e + b^D D + b^{DL} D_L, \qquad \text{(PSCP)}$$

subject to

$$x_{j,t} \le \sum_{q=0}^{t-k_t} x_{i,q} \quad \forall (c_i, c_j) \in E,\ t = 1 \dots S^{max}$$
$$where\ k_t = \begin{cases} 3\ if\ t\ mod\ 5 = 0 \\ 1\ else, \end{cases} \qquad \text{(A.1)}$$

$$x_{j,t} \le \sum_{q=0}^{t-k_t} x_{i,q} + x_{i,t} \quad \forall (c_i, c_j) \in E',\ t = 1 \dots S^{max},$$
$$k_t = \begin{cases} 3\ if\ t\ mod\ 5 = 0 \\ 1\ else, \end{cases} \qquad \text{(A.2)}$$

$$x_{i,t} \le \frac{1}{4} \sum_{j \in L_4} \sum_{q=0}^{t-k_t} x_{j,q} \quad \forall i \in L_5, \quad \forall t = 1, \dots S^{max},$$
$$k_t = \begin{cases} 3\ if\ t\ mod\ 5 = 0 \\ 1\ else, \end{cases} \qquad \text{(A.3)}$$

$$x_{i,t} \le \frac{1}{4} \sum_{j \in L_5} \sum_{q=0}^{t-k_t} x_{j,q} \quad \forall i \in L_6, \quad \forall t = 1, \dots S^{max},$$
$$k_t = \begin{cases} 3\ if\ t\ mod\ 5 = 0 \\ 1\ else, \end{cases} \qquad \text{(A.4)}$$

$$x_{i,t} \le \frac{1}{|L_4|} \sum_{j \in L_4} \sum_{q=0}^{t-k_t} x_{j,q} \quad \forall i \in L_6, \quad \forall t = 1, \dots S^{max},$$
$$k_t = \begin{cases} 3\ if\ t\ mod\ 5 = 0 \\ 1\ else, \end{cases} \qquad \text{(A.5)}$$

$$z_i \ge \frac{1}{|\Delta_i|} \sum_{j \in \Delta_i} x_j \quad \forall i = 1 \dots \omega, \qquad \text{(A.6)}$$

$$z_i \le \sum_{j \in \Delta_i} x_j \quad \forall i = 1 \dots \omega, \qquad \text{(A.7)}$$

$$\sum_{i=1}^{\omega} z_i \ge N^d, \qquad \text{(A.8)}$$

$$z_i \in \{0,1\}\ i = 1 \dots \omega,\ x_i \in \{0,1\},\ i = 1, \dots N,$$
$$x_{i,t} \in \{0,1\},\ i = 1, \dots N,\ t = 0, \dots S^{max}, \qquad \text{(A.9)}$$

$$x_{i,t} \le o_{i,t} \quad \forall i \in \{1, \dots, N\},\ t = 1, \dots S^{max}, \qquad \text{(A.10)}$$

$$\sum_{i=1}^{N} \overline{c}_i x_i \ge T^c\ , \qquad \text{(A.11)}$$

$$\sum_{t=0}^{S^{max}} x_{i,t} = x_i \quad \forall i = 1, \dots N, \qquad \text{(A.12)}$$

$$\sum_{i \in LE} \overline{c}_i x_i \ge L^c, \qquad \text{(A.13)}$$

$$\sum_{i=1}^{N} \overline{c}_i x_{i,t} \le C^{max} \quad \forall t = 1, \dots S^{max}, \qquad \text{(A.14)}$$

$$\sum_{i \in R_j} x_i \ge r_j \quad \forall j = 1, \dots k, \qquad \text{(A.15)}$$

$$x_i = 1 \quad \forall i \in S^d, \qquad \text{(A.16)}$$

$$x_{i,t} = 1 \quad \forall (i,t) \in I^T, \qquad \text{(A.17)}$$

$$x_i = 0 \quad \forall i \in S^u, \quad x_{i,t} = 0 \quad \forall (i,t) \in U^T, \qquad \text{(A.18)}$$

$$x_{j,t} \le \sum_{q=0}^{t-k_t} x_{i,q} + 1 - x_i,\ (c_i, c_j) \in E_s\ t = 1 \dots S^{max}$$
$$k_t = \begin{cases} 3\ if\ t\ mod\ 5 = 0 \\ 1\ else, \end{cases} \qquad \text{(A.19)}$$

$$\sum_{i=1}^{N} x_{i,t} \le T^{max}, \quad \forall t = 1 \dots S^{max}, \qquad \text{(A.20)}$$

$$\sum_{\substack{i=1 \\ i \ne \theta}}^{N} x_{i,t} \le (\Theta^{max} - 1) x_{\theta,t} + M(1 - x_{\theta,t}) \quad \forall t = 1, \dots S^{max}, \qquad \text{(A.21)}$$

$$\sum_{t=1}^{S^{max}} t x_{i,t} \le D \quad \forall i = 1, \dots N, \qquad \text{(A.22)}$$

$$D^L \ge \sum_{i=1}^{N} \overline{d}_i x_{i,t} \quad \forall t \in S, \qquad \text{(A.23)}$$

$$G^e = \sum_{i \in A} \hat{g}_i x_i, \qquad \text{(A.24)}$$

$$\sum_{i \in A} \overline{c}_i x_i \le \hat{C}^{rem}. \qquad \text{(A.24')}$$

In the following, we analyze each of the constraints.

Problem Constraints

Prerequisite and Corequisite Constraints. Given the definitions in Tables A.1 and A.2, we model the prerequisite constraints for the scheduling of courses as follows:

$$x_{j,t} \le \sum_{q=0}^{t-k_t} x_{i,q} \quad \forall (c_i, c_j) \in E, \ t = 1 \ldots S^{max}$$
$$where\ k_t = \begin{cases} 3\ if\ t\ mod\ 5 = 0 \\ 1\ else, \end{cases} \tag{A.1}$$

where $x\ mod\ y$ in Equation (A.1) denotes the remainder of the integer division of the two natural numbers x, y. The meaning of the Constraints (A.1) is that $x_{j,t}$ cannot be one unless the student has taken course c_i at any term $q = 0, 1, \ldots t - k_t$, where $k_t = 1$ if t is a "normal" term and $k_t = 3$ if t is a summer term, in which case the student must have passed course c_i at least by the previous spring term.

Extension to Complex Prerequisite Constraints. At ACG, prerequisite constraints can be a lot more complex than what was just described previously. There are courses for which students can register if they have completed one from a chosen set of other classes. Such disjunctive constraints are particularly useful when a degree program undergoes changes and a course is "phased out" in favor of another new course. In general, such a disjunctive constraint of prerequisite courses, where completion of any course $c_m,\ m \in M_j$ among a set of courses suffices for registering for course c_j, can be modeled via the following constraint:

$$x_{j,t} \le \sum_{m \in M_j} \sum_{q=0}^{t-k_t} x_{m,q}\ t = 1 \ldots S^{max}\ where\ k_t = \begin{cases} 3\ if\ t\ mod\ 5 = 0 \\ 1\ else. \end{cases} \tag{A.1′}$$

There are cases of courses that have even more complex prerequisite constraints: course ITC3287, "Advanced Object Oriented and Functional Programming," lists, as its prerequisites, the following: (a) ITC2088, "Introduction to Programming," and (b) ITC2197, "Object Oriented Programming Techniques," or ITC3234, "Object Oriented Programming." This set of constraints is a conjunction of disjunctions: the student must certainly have passed ITC2088 and must have taken at least one of ITC2197 or ITC3234. This conjunction is easily written as the following two constraints:

$$x_{3287,t} \le \sum_{q=0}^{t-k_t} x_{2088,q}\ where\ k_t = \begin{cases} 3\ if\ t\ mod\ 5 = 0 \\ 1\ else, \end{cases}$$

$$x_{3287,t} \le \sum_{q=0}^{t-k_t} (x_{2197,q} + x_{3234,q})\ where\ k_t = \begin{cases} 3\ if\ t\ mod\ 5 = 0 \\ 1\ else. \end{cases}$$

In general, any set of prerequisite constraints at ACG, in its most general form, is written as a disjunction of conjunctions (DNF), which can automatically be converted to a conjunctive normal form (CNF) that is then modeled as a number of simultaneous constraints that must hold for each conjunct in the CNF, and each constraint is then of the form (A.1′).

Similarly, the corequisite constraints for the scheduling of the courses are as follows:

$$x_{j,t} \le \sum_{q=0}^{t-k_t} x_{i,q} + x_{i,t} \quad \forall (c_i, c_j) \in E', \ t = 1 \ldots S^{max},$$
$$k_t = \begin{cases} 3\ if\ t\ mod\ 5 = 0 \\ 1\ else. \end{cases} \tag{A.2}$$

Constraints (A.2) are essentially the same as (A.1), but now the student is allowed to take course c_j in term t as long as course c_i has been passed until term $t - k_t$ or if $x_{i,t} = 1$, which means that course c_i is taken concurrently with c_j in the same term. There are no complex corequisite constraints currently at ACG.

Level-*k* Constraints. We model the Level-5 constraints using the binary $x_{i,t}$ variables as follows:

$$x_{i,t} \le \frac{1}{4} \sum_{j \in L_4} \sum_{q=0}^{t-k_t} x_{j,q} \quad \forall i \in L_5, \quad \forall t = 1, \ldots S^{max},$$
$$k_t = \begin{cases} 3\ if\ t\ mod\ 5 = 0 \\ 1\ else. \end{cases} \tag{A.3}$$

The meaning of Constraints (A.3) is this: until the term t for which the variables $x_{j,q},\ j \in L_4$ sum to at least four between terms zero and $t - k_t$, the constraint is active, and because the right-hand side (RHS) of the constraint is less than one, it forces the binary variables $x_{i,t}$ to stay at zero, preventing the student from taking any Level-5 course in these early terms; afterward, however, the constraint simply becomes inactive, allowing the student to take the course at this or any later term if they want.

Similarly, the Level-6 constraints are modeled as follows:

$$x_{i,t} \le \frac{1}{4} \sum_{j \in L_5} \sum_{q=0}^{t-k_t} x_{j,q} \quad \forall i \in L_6, \quad \forall t = 1, \ldots S^{max},$$
$$k_t = \begin{cases} 3\ if\ t\ mod\ 5 = 0 \\ 1\ else, \end{cases} \tag{A.4}$$

$$x_{i,t} \le \frac{1}{|L_4|} \sum_{j \in L_4} \sum_{q=0}^{t-k_t} x_{j,q} \quad \forall i \in L_6, \quad \forall t = 1, \ldots S^{max},$$
$$k_t = \begin{cases} 3\ if\ t\ mod\ 5 = 0 \\ 1\ else. \end{cases} \tag{A.5}$$

The constraints (A.4) ask for at least four Level-5 courses to be completed before a Level-6 class becomes available for a student; Constraints (A.5) require that all Level-4 courses are completed before a Level-6 class becomes available to the student.

LE Diversity Constraints. Some programs (other than IT and CYN) specify that the student must complete a certain number of credits from LE courses, but with the extra requirement that the courses the student takes are from at least $N^d > 1$ different disciplines, where the discipline of a course is defined by the program offering it so that, for

example, course ITC1070, "Fundamentals of Information Technology," belongs to the IT discipline, but the course PS1000, "Psychology as a Natural Science," belongs to the psychology discipline. This requirement, when present, is modeled as follows: we introduce a new binary variable for each different discipline that becomes one if the student takes at least one course from it and zero otherwise; we then require that the sum of all these new binary variables is at least equal to N^d. Consider, therefore, the nonempty, pairwise disjunct sets $\Delta_i \subseteq LE,\ i = 1,\dots\omega$ containing the courses that belong to each of the ω different disciplines available in the set LE so that $\cup_{i=1,\dots\omega}\Delta_i = LE$. We require that

$$z_i \geq \frac{1}{|\Delta_i|}\sum_{j\in\Delta_i} x_j \ i = 1\dots\omega, \tag{A.6}$$

$$z_i \leq \sum_{j\in\Delta_i} x_j \ i = 1\dots\omega, \tag{A.7}$$

$$\sum_{i=1}^{\omega} z_i \geq N^d, \tag{A.8}$$

$$z_i \in \{0,1\}\ i = 1\dots\omega,\ x_i \in \{0,1\},\ x_{i,t} \in \{0,1\},\ i = 1\dots N,\ t = 0\dots S^{max}. \tag{A.9}$$

Constraints (A.9) also enforce the binary nature of the variables $x_i,\ x_{i,t}$.

Constraints (A.7) force the binary variable z_i to be zero when no course from the discipline Δ_i has been selected, and Constraints (A.6) force it to be one if at least one course from the discipline is selected. Constraint (A.8) ensures the required diversity of disciplines is enforced.

Availability Constraints. Given the parameters $o_{i,t}$ in Table A.1, the following constraints must hold to ensure that a student cannot register for a course at a time when the course is not offered by the college:

$$x_{i,t} \leq o_{i,t} \ \ \forall i \in \{1,\dots,N\},\ t = 1,\dots S^{max}. \tag{A.10}$$

Credit Constraints. The following constraint ensures that the min required number of credits are taken:

$$\sum_{i=1}^{N} \overline{c}_i x_i \geq T^c\ . \tag{A.11}$$

The value of the variable x_i is simply the sum of the variables $x_{i,t}$ over all terms

$$\sum_{t=0}^{S^{max}} x_{i,t} = x_i \ \ \forall i = 1,\dots N. \tag{A.12}$$

There must be a min number of credits coming from liberal education (LE) courses:

$$\sum_{i\in LE} \overline{c}_i x_i \geq L^c. \tag{A.13}$$

A student cannot take more than a threshold number of credits per term:

$$\sum_{i=1}^{N} \overline{c}_i x_{i,t} \ \leq\ C^{max} \ \ \forall t = 1,\dots S^{max}. \tag{A.14}$$

Concentration Area Constraints. Given a chosen concentration area subsets $R_1,\dots R_k$, the students must select at least a certain number (r_j) of courses from these sets:

$$\sum_{i\in R_j} x_i \geq r_j \ \ \forall j = 1,\dots k. \tag{A.15}$$

Optional User Desire Constraints. Students may specify their desire to take certain courses in a user-specified set S^d:

$$x_i = 1 \ \ \forall i \in S^d. \tag{A.16}$$

They may also specify their desire to take a certain course c_i at a specific term t for a set I^T of such tuples (Table A.1):

$$x_{i,t} = 1 \ \ \forall (i,t) \in I^T. \tag{A.17}$$

Vice versa, if a student does not want to take any courses from the set S^u, or simply during certain terms specified in a set of tuples $(i,t) \in U^T$ (Table A.1), this is enforced by the following constraints:

$$x_i = 0 \ \ \forall i \in S^u,\ \ x_{i,t} = 0 \ \ \forall (i,t) \in U^T\ . \tag{A.18}$$

Soft-Order Constraints. Soft-order precedence constraints for courses $(c_i, c_j) \in E_s$ are modeled as follows:

$$x_{j,t} \leq \sum_{q=0}^{t-k_t} x_{i,q} + 1 - x_i,\ (c_i,c_j) \in E_s\ t = 1\dots S^{max}$$

$$k_t = \begin{cases} 3 \ if\ t\ mod\ 5 = 0 \\ 1 \ else. \end{cases} \tag{A.19}$$

If the user does not take the course c_i, then $x_i = 0$, and the constraint becomes inactive; if they have already taken course c_i in the past, then $x_i = 1,\ x_{i,0} = 1$, and the constraint is inactive, allowing the student to take (or not to take) the course c_j at any point in time; if, however, the user does take (in a future term) course c_i, then the constraint asks that, if they take course c_j, they take it after they take c_i. Furthermore, if the user has already passed course c_j, then the variable $x_{j,0} = 1$ and $x_{j,t} = 0 \ \ \forall t = 1,\dots S^{max}$, and the constraints (A.19) are simply inactive regardless of whether the student takes course c_i in the future.

We enforce a plan with no more than a max number of courses per term T^{max}:

$$\sum_{i=1}^{N} x_{i,t} \leq T^{max},\ \ \forall t = 1\dots S^{max}. \tag{A.20}$$

The students may also optionally indicate a max number of courses $\Theta^{max} \geq 1$ to register for during the same term that they register for their BSc thesis, which is denoted by c_θ.

$$\sum_{\substack{i=1 \\ i\neq\theta}}^{N} x_{i,t} \leq (\Theta^{max} - 1)x_{\theta,t} + M(1 - x_{\theta,t}) \ \ \forall t = 1,\dots S^{max}, \tag{A.21}$$

where M denotes any number greater than or equal to the largest number of courses a student is allowed to take per term (C^{max} is a valid candidate because every course is always worth at least one credit).

Future Extension With Timetabling Constraints. Assume that weekly schedules of all future courses are scheduled in advance so that we know for each course c to be offered in

term t the sections $s^t_{c,1} \ldots s^t_{c,N_{c,t}}$ that are going to be offered for the course and their timing every week (e.g., Monday to Wednesday, 9:00–9:50). Given this, we can know for every two course sections $s^t_{c,j}$, $s^t_{c',j'}$ whether they overlap in time so that we have a predicate function $L(s^t_{c,j}, s^t_{c',j'})$ that becomes true if and only if the two course sections overlap in time. By introducing extra binary variables $x_{c,t,j}\ j = 1 \ldots N_{c,t}\ \forall c, t$ that represent the exact section of course c for which the student will register in term t, we require that the following extra constraints (L1) and (L2) also hold:

$$\sum_{j=1}^{N_{c,t}} x_{c,t,j} = x_{c,t} \quad \forall c = 1, \ldots N,\ t = 1, \ldots S^{max}, \tag{L1}$$

$$x_{c,t,j} + x_{c',t,j'} \leq 1 \quad \forall c, j, c', j', t : L(s^t_{c,j}, s^t_{c',j'}) = true. \tag{L2}$$

The first constraint (L1) provides the relation between the assignment variables $x_{c,t}$ and the fine-grain section assignment variables $x_{c,t,j}$, and the second constraint (L2) simply forbids the student from being assigned to two course sections that overlap in the weekly time schedule.

Objective Functions. The first objective, namely creating a plan with min length of study, is as follows:

$$\min D,$$

subject to the extra constraints:

$$\sum_{t=1}^{S^{max}} t x_{i,t} \leq D \quad \forall i = 1, \ldots N. \tag{A.22}$$

The second objective is to create plans where the total academic difficulty over any term is minimized. Written as a mini-max problem, therefore, the objective function becomes

$$\min_x \max_{t \in S} \sum_{i=1}^{N} \bar{d}_i x_{i,t},$$

where the set S in the equation represents all terms during which the student takes at least one course.

This objective function is written as a mini-max problem; we convert it to a mixed-integer linear program problem as follows:

$$\min D^L,$$

subject to the extra constraints

$$D^L \geq \sum_{i=1}^{N} \bar{d}_i x_{i,t} \quad \forall t \in S. \tag{A.23}$$

Regarding the objective of maximizing the expected student overall GPA upon graduation, the objective to optimize becomes

$$\max \frac{\sum_{\substack{i=1 \\ i \in A}}^{N} \hat{g}_i x_i}{\hat{N}},$$

where A is the set of all courses open for the student to take, for which we have an estimate $\hat{g}$ of the grade the student will get that is above a predefined threshold $g^{thres} > 0$, with the threshold being set to 2.5 for a final grade that ranges between zero and four. This threshold prevents proposing low-predicted-score courses to the student just because we lack estimates for other courses. $\hat{N}$ is the total number of courses required for graduation from the program. To bring the aforementioned objective in MILP format, given that $\hat{N}$ is a constant number, we rewrite it as follows, considering that at any given time, the student must take a known number of credits (say $\hat{C}^{rem} > 0$) to graduate:

$$\max G^e,$$

subject to the extra constraints

$$G^e = \sum_{i \in A} \hat{g}_i x_i, \tag{A.24}$$

$$\sum_{i \in A} \bar{c}_i x_i \leq \hat{C}^{rem}. \tag{A.24'}$$

The maximization objective, together with Constraints (A.24) and (A.24′), aims to maximize the expected GPA of the student given that student's past record and constitute a completely personalized approach to planning for student success. Notice that the maximization problem by itself is a simple knapsack problem.

Finally, the overall objective function that the model minimizes is the following:

$$C(D, D^L, G^e, x, z) = b^G G^e + b^D D + b^{DL} D^L, \tag{A.25}$$

subject to Constraints (A.1)–(A.24′).

Finally, the three types of rules produced by the QARMA algorithm that were described while discussing the use of data mining techniques to produce study plans whose goal is to maximize the expected student GPA are exactly formulated like this:

$$grade(c_i) \geq v_i \rightarrow grade(c_j) \geq v_j,$$

$$grade(c_i) \geq v_i \wedge grade(c_j) \geq v_j \rightarrow grade(c_k) \geq v_k,$$

$$grade(c_i) \geq v_i \wedge grade(c_j) \geq v_j \wedge grade(c_k) \geq v_k \rightarrow grade(c_m) \geq v_m.$$

Appendix B. Proof of Theorem 1

Theorem 1. *The personalized student course plan (PSCP) problem is NP-hard.*

Proof. We prove the theorem by reducing the problem to the NP-complete number partitioning problem. The number partitioning problem asks whether a set of numbers can be partitioned in two sets so that the sums of numbers of the two sets are equal. Consider an instance of the number partitioning problem (NP), with numbers $n_1, \ldots n_k$, and without loss of generality, assume that $\sum_{i=1}^{k} n_i$ is even (otherwise, the (NP) problem is trivially impossible). We will construct an instance of the decision version (yes/no) of the (PSCP), the solution of which solves the just-mentioned (NP) problem instance. The decision version of the (PSCP) problem asks whether there is a course plan that only requires two terms to complete.

We create a program with exactly k courses $c_1, \ldots c_k$ such that the credits of course c_i are exactly $\bar{c}_i = n_i,\ i = 1 \ldots k$, all of which are mandatory for the student to graduate. We further require that none of the k courses have any prerequisites or corequisites, all course difficulty levels are zero, and all courses are offered in all terms. We set the max number of credits a student can register for in any term to be $\left\lfloor \frac{\sum_{i=1}^{k} n_i}{2} \right\rfloor = \frac{1}{2}\sum_{i=1}^{k} n_i$. The student has no particular wishes regarding courses to take or not to take (they have to take all of them), and there is no explicit bound on how many courses the student can take per term. We set the values of the coefficients $b^G = b^{DL} = 0,\ b^D = 1$. Notice that the (PSCP) instance also takes a max number of terms (S^{max}) as its input, which we can set to any sufficiently large value, although, here, we could set $S^{max} = 2$, and then the decision version is simply whether there exists a feasible course plan.

Now, if the optimal solution value to the (PSCP) problem instance we created is exactly two, then the answer to the (NP) problem instance we created is "yes" (the numbers can be partitioned in two sets that sum to the same number); otherwise, the answer to the (NP) problem is "no." It is easy to see that the opposite direction also holds; that is, if the answer to the (NP) problem is "yes," then the optimal solution value to our constructed (PSCP) problem is two; otherwise, the answer is strictly greater than two. The NP-hardness of the (PSCP) problem now trivially follows from the NP-completeness of the (NP) problem. Q.E.D.

Endnotes

[1] See https://www.kalena.es, https://www.roverd.com, and https://www.ofcourse.org.

[2] See www.prepler.com.

[3] See https://www.scipopt.org.

References

Babaei H, Karimpur J, Hadidi A (2015) A survey of approaches for university course timetabling problem. *Comput. Indust. Engrg.* 86:43–59.

Bowman RA (2021) Developing optimal student plans of study. *INFORMS J. Appl. Analytics* 51(6):409–421.

Castro C, Manzano S (2001) Variable and value ordering when solving balanced academic curriculum problems. *6th Workshop ERCIM WG on Constraints* (ERCIM Constraints, Prague, Czech Republic).

Ceschia S, di Gaspero L, Schaerf A (2023) Educational timetabling: Problems, benchmarks, and state-of-the-art results. *Eur. J. Oper. Res.* 308(1):1–18.

Chiarandini M (2012) The balanced academic curriculum problem revisited. *J. Heuristics* 18:119–148.

Christou IT (2011) *Quantitative Methods in Supply Chain Management: Models and Algorithms* (Springer, London).

Christou IT (2019) Avoiding the hay for the needle in the stack: Online rule pruning in rare events detection. *IEEE Internat. Sympos. Wireless Comm. Systems (ISWCS)* (Institute of Electrical and Electronics Engineers, Piscataway, NJ).

Christou IT, Amolochitis E, Tan Z-H (2018) A parallel/distributed algorithmic framework for mining all quantitative association rules. Preprint, submitted April 18, https://arxiv.org/abs/1804.06764.

Christou IT, Kefalakis N, Soldatos JK, Despotopoulou A-M (2022) End-to-end industrial IoT platform for Quality 4.0 applications. *Comput. Indust.* 137:103591.

Comm CL, Mathaisel DFX (1988) College course scheduling: A market for computer software support. *J. Res. Comput. Ed.* 21(2):187–195.

Garcia C (2019) Practice summary: Managing capacity at the University of Mary Washington's College of Business. *INFORMS J. Appl. Analytics* 49(2):167–171.

Gonzalez G, Richards C, Newman A (2018) Optimal course scheduling for United States Air Force Academy cadets. *Interfaces* 48(3):217–234.

Goodfellow I, Pouget-Abadie J, Mirza M, Xu B, Warde-Farley D, Ozair S, Courville A, Bengio Y (2014) Generative adversarial networks. Preprint, submitted June 10, https://arxiv.org/abs/1406.2661.

Gurobi Optimization LLC (2022a) Gurobi Optimizer reference manual. Accessed August 2022, https://www.gurobi.com.

Gurobi Optimization LLC (2022b) Gurobi Java interface. Accessed August 17, 2023, https://support.gurobi.com/hc/en-us/articles/14799677517585-Getting-Started-with-Gurobi-Optimizer.

Gutierrez-Rojas D, Christou IT, Dantas D, Narayanan A, Nardelli PHJ, Yang Y (2022) Performance evaluation of machine learning for fault selection in power transmission lines. *Knowledge Inform. Systems* 64(3):859–883.

Heileman GE, Thompson-Arjona WG, Abar O, Free HW (2019) Does curricular complexity imply program quality? *Proc. ASEE Annual Conf.* (American Society for Engineering Education, Washington, DC).

Hnich B, Kızıltan Z, Walsh T (2002) Modelling a balanced academic curriculum problem. *Proc. CPAIOR'02 (Kent, UK)*, 121–131.

Kassa BA (2015) Implementing a class-scheduling system at the College of Business and Economics of Bahir Dar University, Ethiopia. *Interfaces* 45(3):203–215.

Lin W-C, Tsai C-F (2020) Missing value imputation: A review and analysis of the literature (2006–2017). *Artificial Intelligence Rev.* 53:1487–1509.

Miranda J (2010) eClasSkeduler: A course scheduling system for the executive education unit at the Universidad de Chile. *Interfaces* 40(3):196–207.

Monette J-N, Schaus P, Zampelli S, Deville Y, Dupont P (2007) A CP approach to the balanced academic curriculum problem. *Proc. 7th Internat. Satellite Workshop CP 2007* (Association for Constraint Programming, France).

Schaerf A (1999) A survey of automated timetabling. *Artificial Intelligence Rev.* 13(2):87–127.

Slim A (2016) Curricular analytics in higher education. Unpublished PhD thesis, Department of Electrical and Computer Engineering, University of New Mexico, Albuquerque, NM.

Strichman O (2017) Near-optimal course scheduling at the Technion. *Interfaces* 47(6):537–554.

Takada K, Miyazawa Y, Yamamoto Y, Imada Y, Tsuruta S, Knauf R (2013) Curriculum optimization by correlation analysis and its validation. Holzinger A, Pasi G, eds. *Human-Computer Interaction and Knowledge Discovery in Complex, Unstructured, Big Data. HCI-KDD 2013. Lecture Notes in Computer Science*, vol. 7947 (Springer, Berlin, Heidelberg).

Thompson-Arjona WG (2019) Curricular optimization: Solving for the optimal student success pathway. Unpublished MSc thesis, Department of Electrical and Computer Engineering, University of Kentucky, Lexington, KY.

Unal YZ, Uysal O (2014) A new mixed integer programming model for curriculum balancing: Application to a Turkish university. *Eur. J. Oper. Res.* 238(1):339–347.

Wolsey LA, Nemhauser G (1988) *Integer and Combinatorial Optimization* (Wiley, Hoboken, NJ).

Verification Letter

"Helena Maragou, Dean of the School of Liberal Arts and Sciences, The American College of Greece—Deree, 6 Gravias Street 153 42, Aghia Paraskevi, Athens, Greece, writes:

"It gives me great pleasure to write this verification letter about our experience with the ACG SCORER software tool,

which is presented in the paper titled "Planning Courses for Student Success at the American College of Greece" by Christou, Vagianou, and Vardoulias. At the School of Liberal Arts and Sciences of the American College of Greece, our advising faculty (department heads and program coordinators) engage on a routine basis in the creation of course planning for each of our students. Because of the number of guidelines set forth by the college, the problem of creating such personalized and workable schedules for our students is a very complex problem that gets ever more complicated with the introduction of more courses and programs our college implements in order to remain at the forefront of knowledge and technology. The introduction of the ACG SCORER software in the Information Technology and Cyber-security and Networks programs has been a very welcome tool that has significantly reduced the time advising faculty need to spend in order to create good schedules for our students; most importantly, it allows for schedules that are not only speedily produced but also provably optimal in the mathematical sense. The tool is furthermore efficient in terms of its capacity to edit the created plans as needed. Overall, we feel that the introduction of this software tool has had a very positive effect on academic operations at our institution and are willing to expand its implementation in other programs in our school in the near future.

Ioannis T. Christou, PhD, holds a Dipl. Ing. in electrical engineering from the National Technical University of Athens (NTUA), Greece; an MSc and PhD in computer sciences from the University of Wisconsin at Madison, United States; and an MBA from the joint Athens MBA graduate program between NTUA and the Athens University of Economics and Business. He is a senior research data scientist at NetCompany-Intrasoft and associate professor at the American College of Greece (ACG). He works in AI/deep learning (DL) and optimization.

Evgenia Vagianou holds a BSc in computer information systems from the American College of Greece, Greece, and an MSc in information technology: knowledge-based systems from the University of Edinburgh, United Kingdom. She is head of the Department of Information Technology. She is involved with the course design in an Erasmus+-funded project. Her research interests focus on security by design of AI-enabled cyber systems, blockchain technology, and AI applications in education.

George Vardoulias received a diploma in electrical and computer engineering from the National Technical University of Athens in 1997 and a PhD from the University of Edinburgh, United Kingdom, in 2001. He is an assistant professor at the American College of Greece and senior research engineer at Intracom Defense. His research interests include coexistence of software-defined radio and software-defined networking technologies in tactical wireless networks and federated learning systems for the defense sector.